

```

from PIL import Image, ImageDraw, ImageFilter
import numpy as np
import matplotlib.pyplot as plt

# Function to create a high-frequency checkerboard pattern
def create_checkerboard(size, square_size):
    img = Image.new('L', (size, size), 255) # White background
    draw = ImageDraw.Draw(img)
    for i in range(0, size, square_size):
        for j in range(0, size, square_size):
            # Alternate black and white squares
            if ((i // square_size) % 2 == 0) == ((j // square_size) % 2 == 0):
                draw.rectangle([(i, j), (i + square_size, j + square_size)], fill=0) # Black
    return img

# Parameters
original_size = 512
square_size = 4 # Small squares to create high frequency
downsampled_size = 128 # Significantly lower resolution

# Create the original high-frequency image
original_image = create_checkerboard(original_size, square_size)

print(f"Original image size: {original_image.size}")

```

Original image size: (512, 512)

```

# 1. Downsample WITHOUT anti-aliasing (using NEAREST which causes severe aliasing)
down_sampled_no_aa = original_image.resize((downsampled_size, downsampled_size), Image.Resampling.NEAREST)

# 2. Downsample WITH anti-aliasing (apply Gaussian blur first, then downsample using a better filter)
# The blur radius should be proportional to the downsampling factor
blur_radius = original_size / downsampled_size / 2 # A common heuristic

# Apply Gaussian blur (low-pass filter)
blurred_image = original_image.filter(ImageFilter.GaussianBlur(radius=blur_radius))

# Downsample the blurred image using a high-quality resampling filter (e.g., LANCZOS)
down_sampled_with_aa = blurred_image.resize((downsampled_size, downsampled_size), Image.Resampling.LANCZOS)

print(f"Downsampled image size: {down_sampled_no_aa.size}")

```

[Toggle Gemini](#)

Downsampled image size: (128, 128)

```

# Display the images
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(original_image, cmap='gray')
plt.title('Original High-Frequency Image')
plt.axis('off')

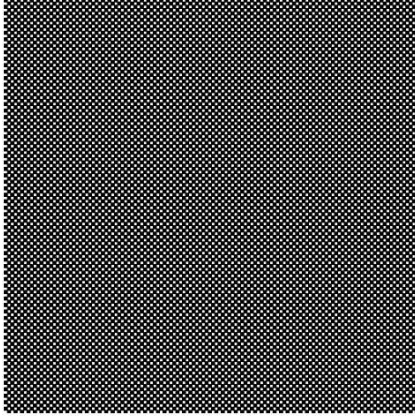
plt.subplot(1, 3, 2)
plt.imshow(down_sampled_no_aa, cmap='gray')
plt.title('Downsampled without Anti-Aliasing (Aliasing visible)')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(down_sampled_with_aa, cmap='gray')
plt.title('Downsampled with Anti-Aliasing (Gaussian Blur + Lanczos)')
plt.axis('off')

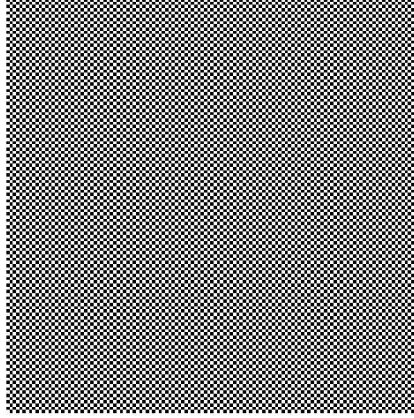
plt.tight_layout()
plt.show()

```

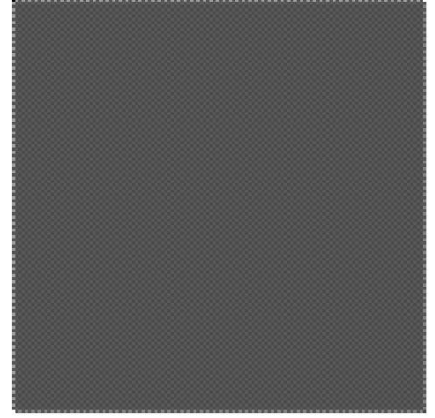
Original High-Frequency Image



Downsampled without Anti-Aliasing (Aliasing visible)



Downsampled with Anti-Aliasing (Gaussian Blur + Lanczos)

[Toggle Gemini](#)